

SPARLEAK: Privacy Leakage from Sparse Attention in LLM Inference on Shared GPUs

Fahao Chen
chenfh@ieee.org
Shandong University
China

Linkang Du
Xi'an Jiaotong University
China
linkangd@xjtu.edu.cn

Jinhao Zhou
Waseda University
Japan
jinhao@suou.waseda.jp

Peng Li
Xi'an Jiaotong University
China
pengli@xjtu.edu.cn

Zhou Su
Xi'an Jiaotong University
China
zhousu@ieee.org

Abstract

Sparse attention is widely used to accelerate long-context inference in modern large language models (LLMs), but its input-dependent execution behavior introduces previously unexplored privacy risks. We identify a new GPU micro-architectural side channel, termed Sparsity-Induced Memory Access (SIMA), which arises from secret-dependent key-value cache access patterns induced by sparse attention.

Based on this observation, we present SPARLEAK, a phase-aware side-channel attack that extracts SIMA traces during LLM inference and enables two practical privacy extractions: query attribute inference from prefill-phase traces and autoregressive response reconstruction from decoding-phase traces. By reconstructing approximate token-level sparsity profiles from page-level observations and applying profiling-based learning, SPARLEAK accurately recovers sensitive information, including user-query attributes and private LLM response content. Extensive evaluation across three LLM architectures, three sparse attention mechanisms, and three privacy-sensitive datasets shows that SPARLEAK achieves average attack success rates of 90.9% for attribute inference and 87.3% for response reconstruction under real-world LLM serving settings, highlighting the significance to account for SIMA leakage when deploying sparse-attention-based LLM systems. We provide anonymized SIMA traces, trained attack models, evaluation scripts, and documentation as artifacts at https://anonymous.4open.science/r/SparLeak_artifacts/.

Keywords: Large Language Models, Sparse Attention, Privacy

1 Introduction

The widespread deployment of large language models (LLMs) [41, 42] has driven the use of increasingly long context to improve generation quality and reduce hallucinations. However, the quadratic cost of attention makes long-context inference computationally expensive. To mitigate this overhead, modern LLM systems increasingly adopt *sparse attention* mechanisms [5, 40, 60], which approximate full attention by selectively attending to a subset of tokens. This trend

is reflected in long-context serving systems such as SGLang [62] and vLLM [18], and is likely to intensify as model scales and context lengths grow. The tokens involved in sparse attention computation are typically identified using activation-based saliency [5, 10, 53, 60], block-level scoring [40], or learned predictors [12, 43, 52]. By pruning irrelevant tokens, sparse attention significantly reduces both prefill-phase and decoding-phase costs.

While designed for efficiency, sparse attention poses a new privacy threat in LLM inference. We find that sparse attention induces secret-dependent memory accesses in the GPU memory hierarchy by dynamically selecting and retrieving Key-Value (KV) tensors based on semantic relevance. These accesses occur internally during GPU execution and are not directly exposed through the LLM service interface. However, they induce observable contention in shared GPU micro-architectural components, such as caches and TLBs. From a security perspective, this effect gives rise to a new side-channel surface rooted in sparsity-induced memory access (SIMA) behavior, which has not been explored by existing works. The SIMA side channel captures how sparse attention externalizes semantic information through data-dependent interactions with the GPU memory hierarchy.

Compared to prior side channels (in Table 1) for exploring privacy leakage during LLM inference based on timing [37, 50, 57, 63], network traffic [28, 48], or embedding cache behavior [1, 13], we find that SIMA side-channel surface is model-intrinsic and spans both prefill and decoding phases, thereby enabling broader and more persistent privacy leakage. These properties reveal an emerging privacy risk inherent to sparse attention mechanisms and motivate timely investigation in modern LLM deployments [38].

Challenges. While the SIMA side channel exposes a promising and principled attack surface, translating this leakage into practical, end-to-end attacks is non-trivial. Successfully exploiting SIMA in real LLM deployments requires overcoming several critical technical challenges. First, SIMA behavior is not directly observable. An attacker cannot see KV accesses or sparsity decisions, and must instead infer them

Side-Channel Attacks	Threat Surface	Extracted Info.	Prefill-Phase	Decoding-Phase	Mitigation	
			Attack	Attack	Prevention	Detection
Time Will Tell [57]	Timing (response time)	Response length	✓	✗	✗	✗
Early Bird [37]	Serving order	KV Cache hit/miss	✓	✗	[6, 26]	[6]
PromptPeek [50]	Timing (time-to-first-token)	KV Cache hit/miss	✓	✗	[6, 26]	[6]
InputSnatch [63]	Timing (time-to-first-token)	KV Cache hit/miss&hit ratio	✓	✗	[6, 26]	[6]
What Was Your Prompt [48]	Network traffic	Token-length sequence	✓	✓	[30, 61]	✗
Whisper Leak [28]	Network traffic&timing	Token-length sequence&timing sequence	✓	✗	[30, 61]	✗
Spill The Beans [1]	CPU shared cache (embedding table)	Embedding access hits	✗	✓	[47]	[19]
I Know What You Said [13]	CPU shared cache (embedding table)	Embedding access hits&access order	✓	✓	[47]	[19]
This work	Sparsity-induced memory access	Sparsity patterns	✓	✓	✗	✗

Table 1. Comparison of representative side-channel attacks that expose privacy leakage in LLM serving. **Prefill-Phase Attack** and **Decoding-Phase Attack** indicate whether an attack can infer sensitive query information or recover generated outputs, respectively. **Mitigation** summarizes existing defense efforts: entries with citations indicate that prior prevention or detection mechanisms have been proposed and can be applied to mitigate the corresponding attack, while ✗ denotes that no effective mitigation has been reported to date.

indirectly from noisy GPU micro-architectural signals. Compounding this difficulty, LLM inference consists of two execution phases, prefill and decoding, with fundamentally different parallelism and access patterns: the prefill phase processes tokens in parallel and thus exposes a cumulative signal, whereas the decoding phase proceeds token by token and exposes a step-wise signal [34, 64]. These phases produce SIMA traces with distinct temporal structures and leakage characteristics, making it necessary to design phase-specific extraction methods. Second, even when micro-architectural leakage can be captured, it is spatially coarse. KV tensors are managed at virtual-memory page granularity [18], and each observed page access aggregates the KV states of many tokens. Consequently, the attacker only observes page-level SIMA traces that collapse multiple token-level accesses into a single event, obscuring fine-grained sparsity structure and discarding much of the information needed to distinguish individual tokens or prompt regions. Third, sparse attention traces encode computational importance rather than explicit lexical content. Recovering meaningful private information therefore requires reconstructing latent access structure and learning the statistical relationship between SIMA traces and high-level semantics or token identities.

Solutions. We present SPARLEAK, the first phase-aware side-channel framework that exploits SIMA behavior during LLM inference. SPARLEAK observes GPU micro-architectural signals and abstracts them into SIMA traces that reflect the input-dependent KV access behavior in prefill and decoding phases, respectively. SPARLEAK then converts these traces into approximate sparsity patterns and uses them as a structural signal to infer sensitive information without accessing the victim’s KV contents. To enable practical exploitation of SIMA leakage, SPARLEAK incorporates the following designs.

First, SPARLEAK introduces two GPU side-channel primitives to extract SIMA traces. It distinguishes prefill and decoding phases solely from GPU execution dynamics: the prefill phase performs highly parallel computation over the input context, whereas the decoding phase proceeds sequentially, generating one token at a time. Exploiting this transition, SPARLEAK recovers cumulative SIMA traces during the prefill phase using an L2-cache-based INVALIDATE+COMPARE primitive, and extracts step-wise SIMA traces during the decoding phase using a TLB-based EVICT+RELOAD primitive.

Second, to overcome spatial aggregation at page granularity, SPARLEAK performs trace reconstruction to infer approximate token-level sparsity profiles. Rather than attempting exact recovery, SPARLEAK exploits spatial locality and distributional regularities in sparse attention. By enforcing statistical consistency with observed SIMA traces, it reconstructs latent access structure sufficient to restore discriminative power for downstream inference.

Finally, SPARLEAK treats reconstructed SIMA traces as a latent structural signal and leverages their statistical correlation with semantic attributes and token identities. Specifically, SPARLEAK learns a model that maps from reconstructed traces to private information through offline profiling. Based on this principle, SPARLEAK realizes two end-to-end inference attacks: Query Attribute Inference (QAI), which infers high-level semantic properties of user prompts from prefill-phase traces, and Autoregressive Token Recovery (ATR), which reconstructs generated tokens from decoding-phase traces. In both cases, the adversary applies a learned mapping at runtime without access to the victim’s model, prompts, or outputs.

Evaluation. We conduct an extensive evaluation of SPARLEAK across multiple widely-adopted LLMs, sparse attention mechanisms, and privacy-sensitive datasets. The results demonstrate that SIMA constitutes a powerful side channel, enabling accurate attack of both query attributes and generated responses. Across a broad range of configurations, SPARLEAK achieves average attack success rates of 90.9% for query attribute inference and 87.3% for autoregressive token

recovery. Notably, even under realistic system noise and model evolution, SPARLEAK retains over 90% of its original attack effectiveness. We further evaluate a potential mitigation by injecting input-independent randomness into sparsity patterns. While this approach reduces attack effectiveness, it does so at the cost of degraded inference quality.

2 Background and Motivation

In this section, we first give background about LLM inference with sparse attention, followed by the motivation of privacy leakage in sparse attention.

2.1 Large Language Models Inference

Attention Mechanism. Modern LLMs are primarily built on the Transformer architecture [45], whose core component is multi-head self-attention. During inference, LLM execution consists of two phases: *prefill* and *decoding*. In the prefill phase, the model processes the entire input sequence in parallel, with computation and memory cost scaling with input length. In the decoding phase, each newly generated token attends to all previously processed tokens. To avoid redundant computation, modern inference engines maintain a key-value (KV) cache that stores the K and V representations of past tokens and reuses them across decoding steps [18, 20].

Sparse attention. To improve inference efficiency, recent work proposes sparse attention mechanisms [10, 40, 43, 53, 60], motivated by the observation that only a small subset of tokens contributes substantially to attention outputs. Rather than attending to all tokens, sparse attention selectively restricts attention computation to a subset of tokens chosen based on the current query or decoding token, reducing computation and memory overhead in both prefill and decoding phases.

Existing sparse attention approaches mainly focus on identifying important tokens without explicitly computing full attention scores. They can be broadly categorized into three classes: (1) *activation-based saliency* methods that infer importance from model activations or residual norms [5, 40, 53, 60]; (2) *block-level scoring* methods that approximate attention at the block or page level and retain tokens from high-scoring blocks [10]; and (3) *learned predictors* that use lightweight models to directly predict important tokens [12, 43, 52].

GPU memory access behavior. As shown in Figure 1, the KV cache of tokens is stored in GPU high-bandwidth memory (HBM) and managed by the GPU virtual memory system, which operates at page granularity [3, 31]. On contemporary NVIDIA GPUs, virtual memory is managed using fixed-size pages that span 64 KB to 2 MB in contiguous virtual address space [3]. In contrast, prior work shows that the KV tensors associated with a single token occupies

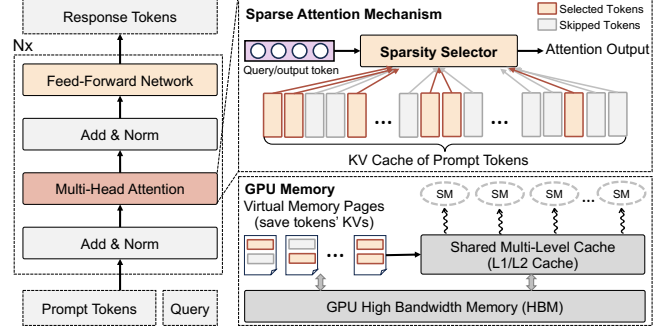


Figure 1. The illustration of sparse attention mechanism in LLM inference and its associated GPU memory access.

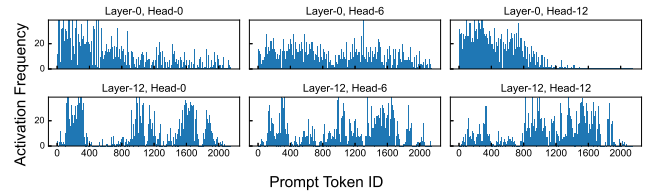


Figure 2. Sparsity pattern in prefill phase. Activation frequency denotes how often a given token is selected by sparse attention during the prefill phase.

only a few kilobytes per layer, typically in the range of 10-30 KB [18]. As a result, a single GPU memory page typically co-locates the KV tensors of multiple tokens.

Under sparse attention, the model first identifies a subset of important tokens for a given token. This selection is ultimately realized at page granularity by the GPU memory subsystem: when a token is selected, the virtual memory pages containing its KV tensors are fetched from HBM into the on-chip cache hierarchy [7]. The fetched KV tensors are then consumed by streaming multiprocessors (SMs) for attention computation, while tokens that are not selected do not trigger page accesses. Consequently, sparse attention induces data-dependent page-level memory access patterns that implicitly aggregate the activities of multiple tokens co-located within each page.

2.2 Motivation

In this section, we characterize sparsity patterns induced by sparse attention during LLM inference. These patterns differ between the prefill and decoding phases, revealing sensitive information about user queries and LLM generated responses, which motivates the SIMA side channel.

Motivation 1: Sparse attention induces distinctive access patterns in the prefill and decoding phases.

Sparse attention mechanisms [5, 40, 60] fundamentally alter the execution behavior of LLM inference. By selectively computing attention over only a subset of tokens, sparse attention induces structured, input-dependent selection of key-value (KV) states, in contrast to dense attention where all tokens are uniformly accessed. While these selection decisions are made at the model level, they directly determine which KV tensors are accessed during inference and how frequently they are reused. From an execution perspective, such non-uniform KV access behavior inevitably translates into data-dependent interactions with the GPU memory hierarchy. Therefore, we characterize the sparsity patterns produced by sparse attention in the prefill and decoding phases, and show that they exhibit qualitatively different structures across the two phases. These differences motivate a phase-aware view of sparsity-induced leakage and SIMA-based side-channel attacks.

We use the LLaMA3-8B [14] model with sparse attention enabled. In our setting, each inference request is formed by concatenating a fixed system prompt and a user-provided query. Unless otherwise stated, the prompt is identical across all motivation experiments and is constructed from an open-source healthcare dataset [35]. All inputs are issued under a fixed system prompt built from an open-source healthcare dataset, with a total prompt length of approximately 2K tokens. During the prefill phase, sparse attention selects the top 10% of prompt tokens for each query token based on approximate attention scores [10].

Prefill-phase sparsity pattern. During the prefill phase, LLM processes entire input context in parallel under sparse attention. Rather than attending uniformly to all context tokens, sparse attention selectively activates a subset of tokens whose KV states contribute to the attention computation. As prefill proceeds, these selective activations form a characteristic sparsity pattern over the input context, reflecting how attention is distributed across different parts of the input.

We visualize this sparsity pattern for an example input in Figure 2. Two observations stand out. First, attention activation is highly skewed: a small subset of tokens is consistently emphasized, while most tokens receive little or no attention. Second, sparsity patterns vary substantially across attention layers and heads. As shown in Figure 2, different layers and heads focus on different parts of the input, suggesting that sparse attention captures complementary semantic information across model components.

Decoding-phase sparsity pattern. In contrast to the prefill phase, decoding phase proceeds autoregressively. At each decoding step, sparse attention selectively activates only a limited subset of context tokens when computing the next-token representation. Consequently, the sparsity pattern observed at a given step reflects which parts of the input context are selectively attended at that moment.

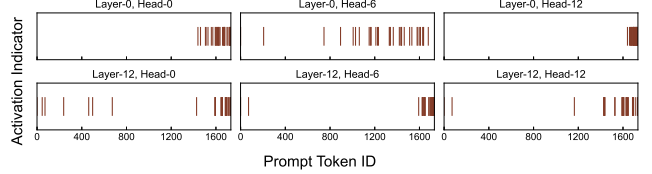


Figure 3. Sparsity pattern at a selected decoding step. Activation indicator is a binary value indicating whether a token is selected by sparse attention at the given decoding step.

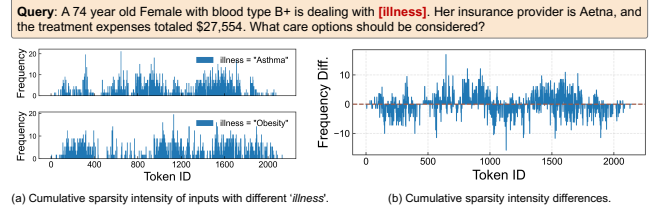


Figure 4. Sparsity patterns of user queries with different attributes. In this example, we change the illness categories in different queries.

We illustrate this behavior by plotting the sparsity pattern at a representative decoding step in Figure 3. As shown, each decoding step activates only a small fraction of the context, confirming that decoding-phase sparsity is highly selective and fine-grained. Similar to the prefill phase, the activated tokens differ markedly across layers and heads, resulting in heterogeneous yet structured sparsity patterns across the LLM.

Motivation 2: *Prefill- and decoding-phase sparsity patterns correlate with sensitive query and responses.*

Building on our analysis of attention sparsity patterns, we first examine whether the sparsity induced during the prefill phase correlates with high-level semantic attributes of user inputs. This question is motivated by prior work showing that side-channel signals derived from model execution can support attribute-level inference even without reconstructing the exact input content [17, 23, 24].

Figure 4 compares prefill-phase sparsity patterns for two inputs that differ only in a single semantic attribute, namely the illness category (e.g., *Obesity* versus *Asthma*), while all other attributes and the system prompt remain identical. As shown in Figure 4(a), the two inputs induce noticeably different sparsity distributions over the same prompt. Certain parts of the prompt are consistently emphasized under one illness category but not the other. To further characterize this effect, Figure 4(b) visualizes the difference between the two sparsity patterns, revealing systematic and non-negligible deviations across the prompt.

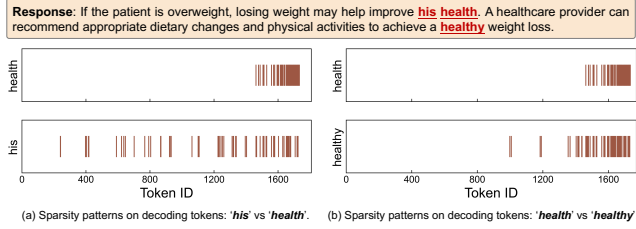


Figure 5. Sparsity patterns of different decoding tokens, including “his”, “health”, and “healthy”.

These results indicate that prefill-phase sparsity patterns are sensitive to high-level semantic attributes of the input. Even when the overall prompt structure and length are fixed, modifying a single semantic attribute reshapes which parts of the prompt are repeatedly attended and how strongly. This observation motivates our *Query Attribute Inference* attack: if an adversary can recover sparsity information corresponding to the prefill phase, it may infer sensitive semantic properties of the user input without reconstructing the exact prompt text.

We then examine whether sparsity patterns during the decoding phase encode information about generated responses. Using the same LLM, prompt, and sparsity configuration, we analyze the attention sparsity pattern at each decoding step. At a given step, the pattern reflects which parts of the prompt are attended when computing the next output token. Figure 5 visualizes representative decoding-phase sparsity patterns induced by different output tokens. As shown in Figure 5(a), distinct tokens (e.g., “his” versus “health”) lead to clearly different sparsity patterns over the prompt. Notably, even semantically related tokens (e.g., “health” and “healthy”) can induce distinguishable sparsity patterns, as illustrated in Figure 5(b). This arises because sparse attention is conditioned not only on the current token but also on the previously generated context, which evolves over the decoding phase and reshapes the attention distribution.

Taken together, these observations show that decoding-phase sparsity patterns are strongly token-dependent. Each generated token induces a distinctive attention footprint shaped jointly by its semantic content and its position in the response sequence. This property underpins our *Autoregressive Token Recovery* attack: if an adversary can infer sparsity patterns across decoding steps, it may progressively distinguish candidate tokens and reconstruct the generated response.

3 Attack Overview

This section presents the threat model, SIMA leakage mechanism, and end-to-end attack workflow.

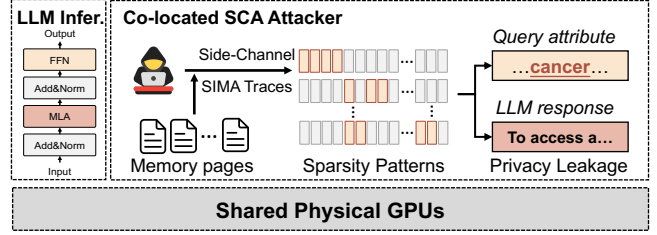


Figure 6. The illustration of the threat model.

3.1 Threat Model

As shown in Figure 6, we consider a realistic LLM serving deployment in which a victim runs sparse-attention-enabled inference on a modern GPU. Specifically, we target a service-style setting in which the victim application uses a public and fixed system-prompt template across requests, while the user query and generated response remain private, as in open-source RAG and agent applications that expose and reuse standardized templates, often with prefix caching. This setting reflects current practice in long-context LLM serving, where sparse attention is increasingly adopted to reduce overhead [10, 40, 53, 60].

Adversary and attack goal. We consider an unprivileged adversary that runs a co-resident spy process on the same physical GPU as the victim, following the standard co-residency threat model in GPU side-channel attacks [44, 63]. The spy process issues GPU probing operations to observe micro-architectural effects induced by the victim’s sparse-attention computation.

The adversary’s goal is to infer sensitive information about both the user query and the LLM’s response via the SIMA side channel. Concretely, the adversary (i) performs *Query Attribute Inference* (QAI) by exploiting cumulative SIMA behavior during the prefill phase, and (ii) conducts *Autoregressive Token Recovery* (ATR) by leveraging step-wise SIMA observations during the decoding phase.

Adversary capabilities. Consistent with prior work [13, 44], we consider an unprivileged adversary co-resident with the victim on the same physical GPU. This setting can arise when a local user runs a CUDA workload on a shared GPU or when a remote tenant’s job is co-scheduled on the same non-partitioned GPU. The adversary requires only user-level CUDA access and monitors memory-related micro-architectural effects, including L2 cache activity and TLB residency across context switches, by measuring the latency of its own probes. This requires no privileged counters, driver modifications, or access to victim memory. The resulting observations expose secret-dependent access patterns induced by sparse attention without directly revealing KV contents or exact victim addresses.

In addition, the adversary has no control over GPU scheduling or exact execution timing. Under the fixed-template serving setting and a fixed model and runtime configuration,

we further assume that the adversary can calibrate contention-based probe mappings for the corresponding prompt-side KV pages offline and reuse them across victim invocations, without accessing KV contents or recovering exact runtime addresses.

Threat surface and practicality. Data-dependent KV accesses induced by sparse attention create the observable GPU memory effects exploited by SPARLEAK. We consider co-resident execution alongside deployed LLM services during normal, uninterrupted operation [62]. Such services maintain long-lived model processes to avoid repeated initialization and continuously serve incoming requests. In this setting, the monitored prompt-side KV allocations are reused across requests, allowing offline page-probe mappings to remain applicable.

3.2 Attack Workflow

As shown in Figure 7, SPARLEAK follows a three-stage attack pipeline that progressively transforms low-level GPU micro-architectural observations into high-level privacy leakage. ① First, it extracts phase-aware SIMA traces from L2 cache and TLB activity, using the KV-cache layout and calibrated probe mappings to distinguish access features across layers and KV heads. It exploits the distinct execution characteristics of prefill and decoding to obtain cumulative traces for the prefill phase and step-wise traces for the decoding phase. ② Second, it reconstructs approximate token-level sparsity patterns from coarse page-level SIMA traces by leveraging the locality and statistical regularity inherent in sparse attention, enabling probabilistic refinement despite page-level aggregation. ③ Finally, it performs sparsity-guided inference attacks that incorporate phase-specific leakage semantics, by training lightweight MLP-based classifiers on the reconstructed sparsity patterns, enabling query attribute inference during the prefill phase and autoregressive token recovery during the decoding phase.

4 Phase-Aware SIMA Trace Extraction

In this section, we first describe how to distinguish the prefill and decoding phases, and then introduce two primitives for SIMA trace extraction in the prefill and decoding phases, respectively.

4.1 Prefill and Decoding Distinction

We distinguish the *prefill* and *decoding* phases based on their distinct GPU execution patterns. During the prefill phase, the model processes the entire input context in a single forward pass, triggering intensive matrix computations over the full context length. Consequently, the prefill phase exhibits a short but concentrated burst of compute-heavy GPU kernels, resulting in high kernel-time density. Here, a kernel refers to a computation routine executed on the GPU, such

as matrix multiplication or attention computation. In contrast, the decoding phase generates tokens iteratively, with each token requiring an incremental forward pass. This produces a long execution trace with a near-periodic structure corresponding to successive token generations.

To locate the transition between these temporal patterns, one feasible approach uses the kernel timeline from Nsight Systems [32] as an independent boundary reference. Nsight was accessible without root privileges in our setup. Using this timeline, GPU activity can be quantified by a kernel-time-based utilization proxy: $util(w) = \frac{\tau}{|w|}$, where τ denotes the cumulative kernel execution time within a fixed window w , and $|w|$ is the window length. Focusing on GEMM kernels provides a cleaner and more stable signal, as they dominate LLM computation. The same transition from concentrated activity to near-periodic execution can also be inferred directly from the intensity and periodicity of the collected L2/TLB traces.

4.2 SIMA Trace Extraction

4.2.1 Prefill-Phase SIMA Trace Extraction. During this phase, the LLM processes the prompt and user query in parallel. Under sparse attention, query tokens selectively access KV states associated with prompt tokens. We define the prefill-phase leakage as a *cumulative SIMA trace*, denoted by $\{c_1, c_2, \dots, c_N\}$, where N is the number of monitored prompt-side KV pages. Each c_i estimates how frequently page p_i is accessed across query-token attention computation during the prefill phase.

Attack Implementation. SPARLEAK leverages the observation that repeated sparse accesses to prompt-side KV pages create measurable replacement pressure in the shared GPU L2 cache. Although the exact GPU L2 replacement policy is undocumented, prior work shows that cache contention produces stable eviction behavior [59]. During offline calibration, for each monitored page p_i , the attacker constructs a small set of cache-line-aligned probe addresses s_i whose L2 eviction behavior is correlated with accesses to p_i , following the methodology of [59].

As illustrated in Figure 8, at runtime the spy invokes `WRITE_CACHE( $s_i$ )` to populate the calibrated probe lines in L2 and applies `READ_CACHE( $s_i$ )` with the `INVALIDATE+COMPARE` primitive while the victim executes the prefill phase under default GPU scheduling. Accesses to page p_i introduce competing cache lines. More frequent accesses therefore produce larger eviction footprints on s_i . By accumulating these footprints over the prefill phase, SPARLEAK estimates c_i for each monitored page and recovers the cumulative SIMA trace, without recovering page contents or exact runtime addresses. While this measurement does not reveal exact access counts, it reliably captures relative page-level sparsity induced by sparse attention, which is sufficient for downstream sparsity reconstruction and inference attacks.

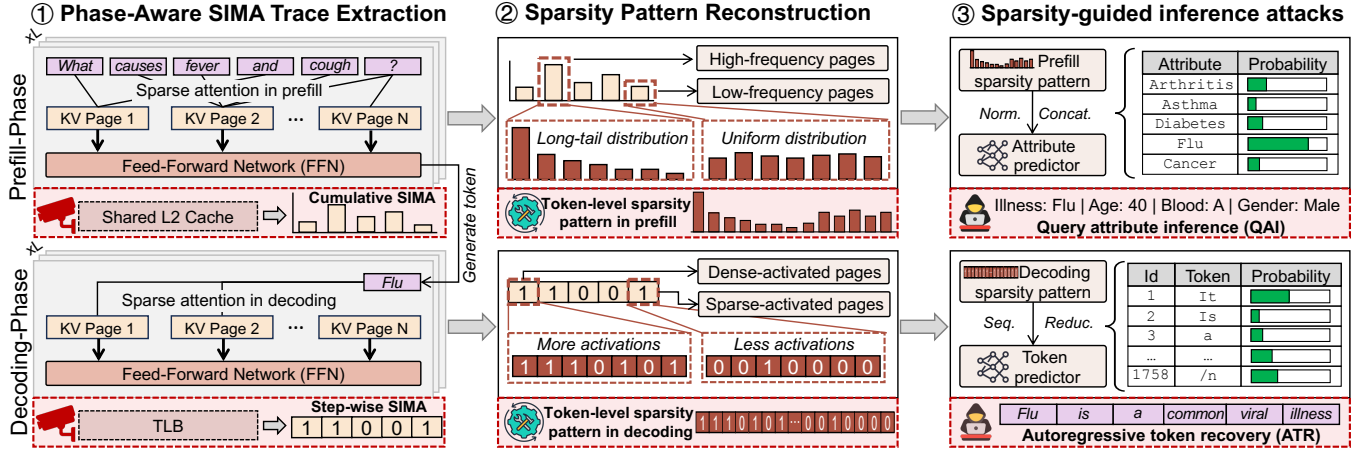


Figure 7. The overview of the attack workflow. SPARLEAK proceeds in three stages: (1) phase-aware extraction of SIMA traces for prefill and decoding phases using different side-channel primitives, (2) reconstruction of token-level sparsity patterns from page-level SIMA traces, and (3) sparsity-guided inference for query attribute inference and autoregressive token recovery.

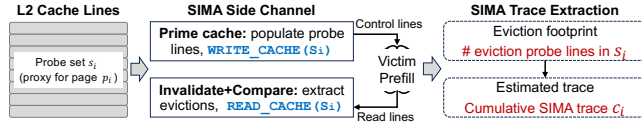


Figure 8. Spy process for estimating page-level access frequency during the prefill phase by repeatedly measuring cache replacement effects induced by sparse-attention KV accesses.

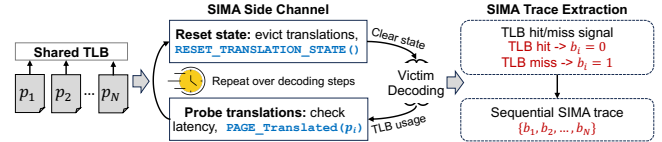


Figure 9. Spy process for recovering the sequential page-level sparsity pattern during the decoding phase. At each decoding step, the spy detects whether KV data from a prompt page are accessed by checking the persistence of its address translation state.

4.2.2 Decoding-Phase SIMA Trace Extraction. During the decoding phase, the LLM generates the response autoregressively, producing one token per decoding step. At step t , LLM computes attention between the newly generated token and a subset of prompt tokens. We characterize the decoding-phase sparsity as a *step-wise SIMA trace* $\{b_1^t, b_2^t, \dots, b_N^t\}$, where $b_i^t \in \{0, 1\}$ indicates whether the KV residing in the page p_i are accessed by sparse attention at step t . Unlike the cumulative trace in the prefill phase, this binary trace captures per-step page participation and evolves across decoding steps.

Attack Implementation. SPARLEAK exploits the GPU Translation Lookaside Buffer (TLB), which caches virtual-to-physical page translations and exposes page-level access footprints under contention. Our design utilizes the GPU TLB `EVICT + RELOAD` primitive demonstrated in prior work [58]. The key idea is to infer whether a monitored prompt-side page p_i is accessed during the decoding phase by measuring the translation latency of an attacker-owned probe page q_i that conflicts with p_i in the TLB, without recovering virtual-page contents or exact victim addresses. Under the fixed model and runtime configuration, the victim pages remain stable

across invocations. The spy therefore calibrates two components offline: for each monitored page p_i , it allocates candidate pages in its own address space and selects a probe page q_i whose translation conflicts with the same or a highly overlapping TLB set as p_i , following the TLB-set calibration methodology of [58]. The spy also estimates the average decoding-step duration from the periodicity of offline probe traces and sets the probing-window length Δ to approximately cover one generation step. In our implementation, one probing round completes within Δ , enabling step-granularity observation.

At runtime, for each window, the spy prepares the calibrated probe-page translations, allows the victim to execute during the window, and then reloads each q_i to measure its translation latency, as shown in Figure 9. If sparse attention accesses p_i , the resulting TLB contention may displace the translation entry of q_i , causing a high-latency reload. SPARLEAK therefore sets $b_i^t = 1$ when the measured latency exceeds a calibrated threshold, and otherwise sets $b_i^t = 0$. Repeating this procedure across monitored pages yields an approximate step-wise SIMA trace $\{b_1^t, \dots, b_N^t\}$.

In practice, a window of length Δ may not perfectly align with a single decoding step due to scheduling jitter or other

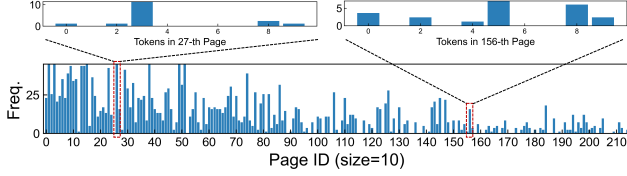


Figure 10. Illustration of page-level and token-level sparsity patterns in the prefill phase.

noise. To tolerate such misalignment, SPARLEAK aggregates measurements from adjacent probing windows and applies majority voting to suppress transient noise.

5 Sparsity Pattern Reconstruction

This section presents reconstruction methods for refining token-level sparsity from extracted SIMA traces, which provide more fine-grained information for subsequent attacks.

5.1 Prefill-Phase Sparsity Reconstruction

Reconstructing token-level sparsity patterns from page-level SIMA traces is challenging because page-level observations compress the activations of multiple tokens into a single value, thereby losing fine-grained intra-page information. A straightforward reconstruction that uniformly distributes the observed accesses across tokens within each page would fail to preserve the highly non-uniform structure of sparse attention. Fortunately, we find that the page-level cumulative access frequency is correlated with the latent token-level activation distribution within the page, providing a useful statistical cue for reconstruction.

To demonstrate this correlation, Figure 10 presents page-level aggregations of prefill-phase sparsity patterns with a page size of 10 tokens. The bottom panel shows the page-level cumulative SIMA trace, while the top panels show the token-level activation frequencies within two representative pages. A clear correlation emerges: pages with higher cumulative frequency tend to exhibit more skewed, long-tailed token-level activation patterns, whereas low-frequency pages show flatter and more uniform activation across tokens. For example, the 27-th page (top-left) lies in a high-frequency region and contains a small number of tokens that dominate activation, while 156-th page (top-right) lies in a low-frequency region and displays an even activation distribution.

This correlation is consistent with how sparse attention selects tokens during the prefill phase. For all query tokens, sparse attention repeatedly reuses only a small subset of highly relevant prompt tokens, while most prompt tokens are selected rarely [51, 65]. Such repeated selection yields a long-tailed distribution of token-level activation frequencies. Since page-level frequency is computed by summing token-level activations within each page, pages that contain

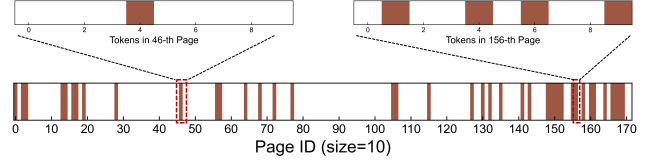


Figure 11. Illustration of page-level and token-level sparsity patterns in the decoding phase.

one or more frequently selected tokens naturally accumulate higher total frequency and appear more concentrated at the token level. Conversely, pages dominated by infrequently selected tokens accumulate lower frequency and consequently appear flatter.

Motivated by this observation, SPARLEAK performs distributional reconstruction to reconstruct an approximate token-level sparsity from page-level SIMA traces. Concretely, SPARLEAK treats each page’s observed frequency as a control signal that determines how concentrated the token-level sparsity distribution should be: higher-frequency pages are modeled with more skewed token participation, while lower-frequency pages are modeled with flatter participation. The reconstructed token-level sparsity is generated in a way that preserves consistency with observed page-level frequency.

5.2 Decoding-Phase Sparsity Reconstruction

The decoding phase introduces a distinct reconstruction challenge. Unlike the cumulative prefill-phase trace, the step-wise SIMA trace provides only a binary observation for each page: an activated page indicates that some token within it is accessed, but does not reveal its intra-page activation pattern. We therefore examine whether the spatial distribution of activated pages at each decoding step provides additional information for reconstruction.

Figure 11 illustrates the relationship between page-level and token-level sparsity during the decoding phase for a representative decoding step (step 5 in this example). We partition the prompt into pages of 10 contiguous tokens and visualize two views: (i) the step-wise page-level SIMA trace observed at decoding time (bottom), and (ii) the corresponding ground-truth token-level activations within two selected pages (top), which are shown only for illustration. An empirical correlation can be observed: pages that are densely activated at the page level during the decoding phase tend to contain multiple activated tokens, whereas pages that are sparsely activated typically contain only a small number of active tokens. For example, the page region around the 156-th page exhibits frequent page-level activation and contains several activated tokens, while the region around the 46-th page is rarely activated and shows only limited token-level activity. Although the exact activation patterns vary across decoding steps and individual tokens, the number of active tokens

within a page closely follows the local density of the page-level SIMA trace.

This behavior is consistent with the locality of sparse attention during the decoding phase. At each decoding step, sparse attention selects only a small set of prompt tokens, and these selections are not uniformly distributed across the prompt. Instead, selected tokens tend to cluster within a few contiguous regions, while most regions receive no activation. The page-level SIMA trace provides a coarse projection of this token-level selection onto page boundaries: clustered token selections lead to dense page-level activation, whereas isolated selections result in sparse page-level signals.

Motivated by this observation, SPARLEAK reconstructs an approximate token-level sparsity pattern by exploiting spatial locality in sparse attention. SPARLEAK uses the local density of page-level activations around a target page as a proxy for the extent of intra-page token participation. Based on this proxy, SPARLEAK instantiates a representative token-level sparsity pattern that is consistent with the observed page-level SIMA trace, while preserving the spatial structure and step-wise dynamics of sparse attention.

6 Sparsity-Guided Inference Attack

In this section, we describe how SPARLEAK infers sensitive information from both user queries and LLM responses.

6.1 Query Attribute Inference

The adversary aims to infer sensitive attributes of a user query, such as health-related semantic categories, consistent with prior inference attacks [27, 33]. SPARLEAK formulates this as supervised classification using reconstructed token-level sparsity profiles from the prefill phase (§5), which capture attention allocation across prompt tokens. During offline profiling, the adversary collects sparsity profiles paired with known attribute labels from its own queries. The classifier learns associations between these structural patterns and semantic attributes, enabling online inference without access to the victim’s query text.

Normalization for scale invariance. Token-level sparsity profiles are affected by query length and the overall amount of attention activity. To prevent these factors from obscuring attribute-specific patterns, we normalize each sparsity profile to remove global scale differences. This emphasizes the relative distribution of sparse attention across prompt tokens, allowing the classifier to focus on structural variations that are more indicative of semantic attributes.

Cross-layer and cross-head feature composition. Sparse attention decisions are made independently across layers and attention heads, and different components may capture complementary aspects of the input. Rather than collapsing these signals prematurely, we retain sparsity features from

all layers and heads and concatenate them into a unified representation. This preserves cross-layer regularities and variability in sparsity behavior, improving the robustness and accuracy of attribute inference.

6.2 Autoregressive Token Recovery

SPARLEAK recovers generated responses token by token using step-wise token-level sparsity profiles reconstructed during decoding (§4.2.2). We formulate recovery as classification at each decoding step. During offline profiling, the adversary pairs step-wise sparsity profiles with known output tokens from its own queries. Features from the monitored layers and KV heads are concatenated to train a token classifier. Online recovery applies this classifier sequentially to reconstructed profiles without accessing victim outputs.

Sequential feature augmentation. Although sparsity profiles at individual decoding steps already contain discriminative information, autoregressive generation exhibits strong temporal dependence. To capture such dependencies without increasing feature dimensionality, we augment each step’s feature representation with aggregated sparsity information from preceding decoding steps. This allows the classifier to exploit temporal regularities in how sparsity patterns evolve as decoding progresses.

Restricted token prediction space. To reduce the complexity and class imbalance of full-vocabulary classification, SPARLEAK constructs a task-specific candidate set from the offline profiling corpus. The set is fixed before online execution and reused within the same task. Online recovery uses SIMA traces to predict tokens within this set, without accessing victim outputs. Supplementary experiments examine whether the gains come from SIMA or the restricted candidate set.

7 Evaluation

7.1 Evaluation Setup

Models. We evaluate three representative LLMs from different model families: LongChat-7B [21], LLaMA3-8B [14], and Qwen3-8B [55]. Each model is configured with three classes of sparse attention mechanisms: (1) *activation-based saliency* [5, 53, 60], which identifies important tokens by comparing intermediate activations; (2) *block-level scoring* [10, 40], which groups tokens into blocks or pages and computes approximate attention scores at the block level; and (3) *learned predictors* [12, 43, 52], which employ lightweight predictors to directly select salient tokens.

Datasets. We use three widely adopted datasets for privacy-sensitive evaluation: Healthcare [35], Financial QA [36], and Legal QA [11]. In the Healthcare dataset, each sample contains only attribute values. We construct LLM inference requests by transforming these attributes into unstructured queries using templates generated by a public model. For the Financial QA and Legal QA datasets, we summarize multiple

Attribute	Signal	LongChat-7B [21]			LLaMA3-8B [14]			Qwen3-8B [55]		
		Act.	Block	Learn.	Act.	Block	Learn.	Act.	Block	Learn.
Illness (60)	O(T)	89.4±1.1	83.3±0.7	99.8±0.1	86.9±0.8	92.1±1.1	99.5±0.4	86.3±0.8	86.8±0.5	98.4±0.3
	O(P+R)	87.3±0.7	81.7±1.1	99.4±0.3	84.5±0.5	90.9±0.6	99.2±0.1	84.4±1.0	85.2±0.2	98.1±0.3
	R(P)	70.4±2.6	66.4±1.2	78.7±0.7	67.1±1.9	66.51±1.4	74.4±1.2	60.3±3.6	62.3±1.8	75.3±1.0
	R(P+R)	85.3±0.4	80.5±0.3	98.8±0.6	84.1±0.6	88.4±0.7	98.7±0.5	81.8±0.4	83.0±0.5	94.7±0.2
Age (8)	O(T)	86.2±0.5	79.7±0.3	99.9±0.1	68.4±1.0	81.3±0.8	99.4±0.3	99.5±0.4	99.1±0.2	99.8±0.1
	O(P+R)	85.3±0.2	77.3±0.4	99.8±0.1	67.3±0.5	79.2±0.4	98.2±0.2	99.4±0.1	98.1±0.2	99.6±0.1
	R(P)	68.3±1.2	58.0±2.0	82.3±1.8	45.2±3.3	59.7±1.4	81.3±0.8	79.8±1.4	81.5±2.2	85.1±1.4
	R(P+R)	82.3±0.4	74.5±0.4	99.8±0.1	62.0±0.9	77.9±0.5	97.1±0.3	97.4±0.1	97.1±0.3	99.4±0.1
Gender (2)	O(T)	98.2±0.2	99.1±0.3	99.8±0.1	94.1±0.3	99.9±0.0	99.7±0.2	99.5±0.4	99.9±0.1	100.0±0.0
	O(P+R)	97.5±0.3	98.7±0.3	99.4±0.3	91.8±0.4	99.8±0.2	99.6±0.1	99.5±0.2	99.6±0.3	100.0±0.0
	R(P)	81.5±0.6	82.3±0.4	83.5±1.3	77.3±1.3	80.6±1.5	83.6±1.6	85.2±1.6	84.7±2.1	85.4±2.8
	R(P+R)	96.4±0.2	96.8±0.4	99.3±0.6	90.6±0.4	99.7±0.2	99.4±0.5	99.3±0.3	99.7±0.1	100.0±0.0
Blood (8)	O(T)	72.3±0.6	93.5±0.3	99.8±0.2	60.4±0.6	95.3±0.5	99.9±0.0	81.4±0.6	99.5±0.4	97.4±0.3
	O(P+R)	71.1±0.4	92.6±0.3	99.8±0.1	59.2±0.4	93.3±0.4	99.9±0.1	80.3±0.4	99.5±0.3	97.2±0.2
	R(P)	49.3±1.1	77.5±0.9	82.2±1.6	37.6±5.5	78.7±0.6	85.6±2.5	60.5±1.5	81.6±1.6	80.3±1.7
	R(P+R)	67.9±0.6	91.5±0.3	99.7±0.1	55.2±0.3	92.0±0.3	99.8±0.1	77.4±0.5	99.1±0.4	95.5±0.2
Entity (70)	O(T)	86.4±0.3	91.3±0.2	93.4±0.4	84.3±0.2	86.1±0.5	89.3±0.2	91.9±0.3	95.4±0.3	97.2±0.4
	O(P+R)	85.0±0.3	90.3±0.4	92.3±0.3	84.0±0.2	84.3±0.2	88.4±0.2	90.7±0.2	93.3±0.3	96.4±0.3
	R(P)	68.1±1.3	71.4±1.1	79.2±0.9	66.6±1.1	69.4±1.0	68.5±1.2	70.4±1.8	80.4±0.5	82.1±2.3
	R(P+R)	84.0±0.3	88.3±0.2	90.3±0.4	80.3±0.5	82.0±0.3	84.9±0.3	88.6±0.3	92.2±0.3	94.8±0.4
Period (4)	O(T)	94.0±0.4	94.3±0.2	97.4±0.3	95.9±0.4	98.3±0.2	99.1±0.2	97.7±0.3	98.5±0.1	99.8±0.1
	O(P+R)	93.2±0.3	92.2±0.2	97.0±0.3	95.1±0.2	97.4±0.2	98.3±0.3	96.1±0.3	97.7±0.2	99.7±0.1
	R(P)	79.6±1.1	77.9±1.0	81.2±1.3	80.5±0.8	82.4±1.7	83.2±1.6	81.4±1.5	79.9±2.5	84.3±1.4
	R(P+R)	92.7±0.2	91.5±0.4	95.8±0.3	93.6±0.2	96.6±0.2	97.9±0.2	95.8±0.2	96.5±0.3	99.4±0.4
Topic (5)	O(T)	93.4±0.3	97.3±0.2	98.6±0.2	91.1±0.3	95.3±0.4	95.3±0.4	89.0±0.2	91.8±0.3	92.5±0.5
	O(P+R)	92.6±0.3	96.9±0.1	96.2±0.3	90.1±0.3	95.0±0.4	94.3±0.3	87.3±0.2	90.2±0.4	91.3±0.2
	R(P)	76.3±1.2	79.5±1.3	81.2±2.2	72.3±1.3	78.2±2.1	79.4±2.3	75.5±2.1	76.3±1.6	77.3±1.7
	R(P+R)	90.3±0.4	94.0±0.3	94.8±0.4	87.8±0.4	92.2±0.4	92.4±0.3	85.8±0.2	88.2±0.3	89.8±0.2
Domain (15)	O(T)	94.1±0.2	94.6±0.4	98.5±0.2	95.8±0.3	97.3±0.4	98.8±0.2	91.2±0.3	91.3±0.3	98.4±0.2
	O(P+R)	93.1±0.3	93.3±0.3	97.1±0.3	94.3±0.3	96.0±0.3	97.2±0.2	90.1±0.3	91.1±0.3	97.2±0.3
	R(P)	79.3±1.0	80.1±1.6	81.3±1.5	79.4±1.8	80.2±1.2	82.1±2.2	72.5±1.5	71.5±1.1	80.2±1.9
	R(P+R)	90.2±0.6	90.8±0.4	95.3±0.4	92.3±0.4	94.9±0.3	95.6±0.3	87.6±0.4	87.8±0.4	95.2±0.2
Role (8)	O(T)	97.9±0.2	97.4±0.4	99.2±0.2	99.8±0.1	99.3±0.2	99.8±0.1	95.9±0.3	98.3±0.3	98.3±0.2
	O(P+R)	96.3±0.2	95.8±0.3	98.4±0.2	99.3±0.0	98.3±0.2	99.1±0.2	94.3±0.2	96.8±0.3	97.2±0.1
	R(P)	79.5±1.9	78.4±1.7	79.3±1.6	79.9±1.0	79.1±1.4	81.1±2.4	75.6±2.0	79.9±0.7	81.3±3.1
	R(P+R)	94.9±0.3	94.8±0.2	96.8±0.3	97.6±0.3	96.9±0.3	97.8±0.4	92.6±0.3	95.4±0.2	95.1±0.3
Location (20)	O(T)	88.8±0.1	89.6±0.3	92.2±0.3	87.7±0.2	85.4±0.2	89.6±0.3	91.8±0.2	93.3±0.3	95.2±0.3
	O(P+R)	87.3±0.2	87.3±0.4	91.4±0.2	86.6±0.3	84.3±0.3	88.0±0.2	90.3±0.3	92.1±0.3	93.9±0.3
	R(P)	69.3±1.6	70.3±1.3	71.5±2.0	69.8±1.2	68.5±1.6	71.4±1.7	72.5±1.8	73.1±1.3	75.2±1.4
	R(P+R)	85.5±0.3	86.9±0.4	88.1±0.2	84.9±0.2	82.3±0.4	86.1±0.4	88.3±0.4	89.7±0.3	91.9±0.3

Table 2. Query attribute inference PASR (%) on Healthcare [35] (Illness, Age, Gender, Blood), Financial QA [36] (Entity, Period, Topic), and Legal QA [11] (Domain, Role, Location). Each entry report the mean $\pm$ 95% confidence interval over 20 runs.

attributes within each query to enable attribute inference attacks. All constructed queries are fed into the deployed LLMs to generate responses, which are subsequently used to perform autoregressive token recovery attacks.

Metrics. We evaluate attack effectiveness separately for the prefill and decoding phases using phase-specific attack success rates. The prefill-phase attack success rate (PASR) is the fraction of correct attribute predictions among queries containing the corresponding attribute. Its denominator may

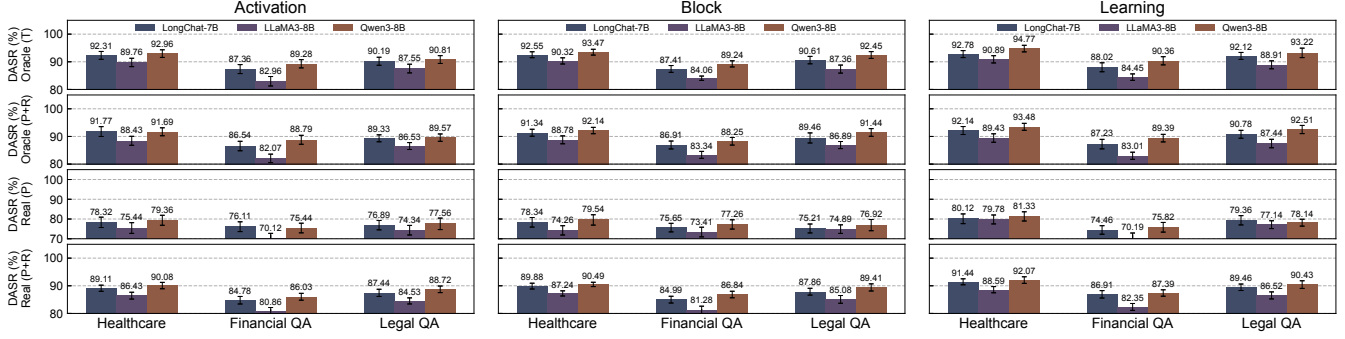


Figure 12. Performance of autoregressive token recovery attacks, measured by DASR.

therefore vary across attributes. The decoding-phase attack success rate (DASR) is the fraction of correctly reconstructed tokens per response, averaged across queries. Ablation studies additionally report the similarity between extracted SIMA traces and ground-truth sparsity patterns.

Environment. We evaluate the performance of SPARLEAK on an NVIDIA L20 GPU (48GB) under Ubuntu 20.04 (Linux 5.15.0), using NVIDIA driver 550.67, CUDA 11.8, PyTorch 2.6.0.

7.2 Attack Performance

Table 2 and Figure 12 report PASR and DASR, respectively. The results distinguish token-level oracle patterns (O(T)), reconstructed oracle page patterns (O(P+R)), raw page-level observations (R(P)), and reconstructed real observations (R(P+R)), where R(P+R) represents the complete end-to-end attack.

Overall effectiveness. In the end-to-end attack, average PASR exceeds 91% across all datasets, while DASR ranges from 84.6% to 89.4%, with every setting exceeding 80%. The high O(T) performance confirms that token-level sparsity patterns inherently encode substantial information about private queries and generated responses. O(P+R) remains within approximately one percentage point of O(T), showing limited information loss from page aggregation after reconstruction. End-to-end performance is only about 2.5 points below the token-level oracle, showing that reconstruction recovers most information from noisy page-level traces.

Impact of attribute complexity. PASR varies across attribute types. Attributes with fewer classes, such as *Gender* in Healthcare, achieve near-perfect accuracy (around 97.9%), whereas those with larger label spaces, such as *Entity* in Financial QA, yield lower PASR (approximately 87.3%), reflecting the difficulty of distinguishing more semantic classes. Decoding DASR is less sensitive to attribute semantics because token recovery reconstructs output sequences. However, Financial QA consistently yields lower DASR than Healthcare and Legal QA, reflecting the greater lexical diversity and structural complexity of financial responses.



Figure 13. The performance of sparsity reconstruction.

Impact of sparse attention mechanisms. Sparse attention mechanisms affect attack performance in both phases. Activation-based methods yield lower prefill PASR (approximately 86.4%) than block-level scoring and learned predictors, with the latter achieving the highest PASR (around 95.5%). Decoding follows a similar trend: activation-based methods yield lower DASR, block-level scoring provides moderate improvements, and learned predictors perform best across datasets and model backbones. These results suggest that more accurate sparsity estimation reduces information loss in SIMA traces, benefiting both attribute inference and token recovery.

Impact of LLM architectures. SPARLEAK achieves average prefill PASRs of approximately 90.6%, 89.4%, and 92.7% on LongChat-7B, LLaMA3-8B, and Qwen3-8B, respectively. In addition, Qwen3-8B consistently achieves the highest decoding DASR across datasets (Figure 12). This consistency suggests that its sparsity patterns across layers and heads are more stable and distinctive, yielding cleaner SIMA signals for attribute inference and token recovery.

7.3 Ablation Study

We evaluate the proposed reconstruction methods by aggregating ground-truth token-level sparsity patterns into pages of 5 to 20 tokens, reflecting practical KV page granularities, and measuring PASR and DASR before and after reconstruction. As shown in Figure 13, coarser page aggregation increasingly obscures token-level information and degrades both attacks without reconstruction. At a page size of 20,

Model	Attribute Inference Attack			Token Recovery Attack			
	Per-query prefill time	#Samples	Cost	Per-token decode time	Avg. length	#Samples	Cost
LongChat-7B [21]	0.62s	5,000	0.86h	0.05s	300	3,000	12.7h
LLaMA3-8B [14]	0.58s	5,000	0.82h	0.06s	300	3,000	15.2h
Qwen3-8B [55]	0.79s	5,000	1.10h	0.07s	500	2,000	19.5h

Table 3. Offline profiling overhead for query attribute inference and autoregressive token recovery attacks.

the Healthcare PASR drops to 57.45% without reconstruction but remains substantially higher when reconstruction is enabled. More ablation studies can be found at the submitted supplementary material.

7.4 Practicality Analysis

We further study the feasibility of SPARLEAK, which depends on (i) whether the offline preparation incurs prohibitive cost, and (ii) whether the online side-channel observations can be conducted unobtrusively in realistic deployment settings with strong stealthiness.

Profiling settings and overhead. Our primary evaluation assumes that the model checkpoint, tokenizer, sparse-attention implementation, and configuration are public. To isolate SIMA leakage from configuration mismatch, attacker runs an instrumented replica on controlled inputs to collect ground-truth sparsity patterns and train predictors. This profiling may run on a separate GPU, while page-probe calibration remains target-specific. We use 5,000 QAI queries per model and 3,000/3,000/2,000 ATR responses for LongChat-7B, LLaMA3-8B, and Qwen3-8B, respectively, with an 8:1:1 sample-disjoint split and task-specific 18-/34-layer residual predictors. As summarized in Table 3, LLM inference dominates the profiling cost: QAI requires one prefill pass per query, whereas ATR records step-wise patterns throughout decoding and therefore scales with response length. Predictor training adds negligible overhead, and this one-time cost is amortized across attacks using same configuration.

When model weights or sparse-attention configurations are unavailable, SPARLEAK instead performs target-interactive profiling. The attacker submits chosen queries while a co-resident spy records the corresponding real page-level SIMA traces; attacker-known attributes and returned tokens provide the QAI and ATR labels. Training directly on these trace-label pairs achieves average PASR and DASR of 90.97% and 86.65%, with maximum reductions of only 1.28 and 1.54 percentage points relative to replica-based profiling. Thus, SPARLEAK remains effective without model or sparsity access, at the cost of one-time same-GPU chosen-query collection.

Stealthiness. We assume an adversary co-located with the victim on the same physical GPU that passively collects side-channel signals during LLM inference. For SPARLEAK to be practical, such measurements must not noticeably perturb the victim’s execution. Our evaluation shows that this requirement is satisfied: enabling the spy process introduces only limited overhead, increasing inference latency by 1–3%

Intensity	Baseline		10%		20%		30%	
	ASR (%)	Rouge	ASR (%)	Rouge	ASR (%)	Rouge	ASR (%)	Rouge
LongChat-7B [21]	89.45	0.43	85.41	0.38	82.09	0.32	76.17	0.21
LLaMA3-8B [14]	90.12	0.48	84.92	0.35	79.85	0.24	72.68	0.16
Qwen3-8B [55]	91.23	0.42	87.28	0.37	81.13	0.35	75.98	0.26

Table 4. Effectiveness of fixed sparsity pattern injection as a mitigation strategy. Mitigation intensity denotes the fraction of sparse tokens that are replaced by a random token set.

during the prefill phase (via L2-cache-based INVALIDATE+COMPARE) and 4–6% during the decoding phase (via TLB-based EVICT + RELOAD), with the worst-case slowdown remaining below 8%. These overheads fall well within the normal latency variation of LLM serving environments, where scheduling and resource contention already induce comparable fluctuations. Consequently, the additional delay introduced by SPARLEAK is indistinguishable from benign system noise, rendering the attack stealthy in realistic deployment scenarios.

7.5 Possible Mitigation

In this section, we discuss potential defenses against the sparsity-induced side channels exploited by SPARLEAK, including model-level mitigations and general side-channel hardening. Their effectiveness must be considered alongside their impact on generation quality and serving efficiency. A direct mitigation reduces the information exposed by sparsity patterns through controlled, input-independent randomness. During inference, an $A\%$ fraction of tokens selected by importance scores is randomly replaced with other tokens in an input-independent manner. This perturbation disrupts the stability of input-dependent sparsity patterns, making SIMA traces less discriminative to an adversary. However, the replacement tokens may be less semantically relevant, weakening the selected context and degrading generation quality, particularly at high perturbation intensities.

We evaluate this mitigation on Financial QA with perturbation intensities ranging from 10% to 30%. We report overall ASR, defined as the average of PASR and DASR, and generation quality measured by ROUGE. Increasing perturbation consistently reduces ASR but also degrades generation quality. For example, at 30% perturbation, LLaMA3-8B’s ROUGE score drops from 0.48 to 0.16. These results show that randomizing sparse selection mitigates leakage, but aggressive perturbation compromises the generation quality.

Restricting fine-grained timers, monitoring interfaces, and memory-management primitives can hinder SPARLEAK by reducing trace fidelity without eliminating secret-dependent accesses. Fixed or page-padded schedules and dummy KV accesses reduce input dependence at the cost of sparse-attention efficiency. Tenant isolation and cache/TLB partitioning can prevent cross-tenant observation but require runtime or hardware support. Detecting repeated probing or anomalous cache/TLB

activity can complement these defenses. Developing low-cost defenses that preserve both model quality and serving efficiency remains future work.

8 Related Work

Privacy leakage in LLMs. Prior work has shown that privacy risks in LLMs arise from algorithm- or API-level exposure [8, 54], including training-data memorization [2, 22, 46], membership inference [9, 15, 29, 49], and prompt attribute inference [4, 16, 25, 39, 56]. These attacks typically rely on observing model outputs or interacting with the LLM through crafted prompts. In contrast, SPARLEAK targets inference-time execution behavior and demonstrates that sparse attention can externalize sensitive information through SIMA side channels in shared GPU components, without any interaction at the API level.

Side-channel attacks on LLMs. Recent studies have revealed that LLM serving systems are vulnerable to various side channels, including timing-based attacks [37, 50, 57, 63], network traffic analysis [28, 48], and CPU cache monitoring [1, 13]. These approaches infer information through externally observable signals such as latency, encrypted traffic patterns, or embedding-table accesses. SPARLEAK differs fundamentally by exploiting memory access behavior intrinsic to sparse attention. This leakage is amplified by sparsity and manifests consistently across both prefill and decoding phases, enabling a unified framework for query attribute inference and autoregressive token recovery.

9 Conclusion

We present SPARLEAK, the first study of micro-architectural privacy leakage in sparse-attention-based LLM serving. Sparse attention produces distinctive sparsity-induced memory access (SIMA) patterns that expose sensitive information through shared GPU resources. SPARLEAK exploits phase-specific SIMA traces for query attribute inference during prefill and autoregressive token recovery during decoding. Evaluations across multiple LLMs, sparse attention mechanisms, and datasets demonstrate its effectiveness and robustness.

References

- [1] Andrew Adiletta and Berk Sunar. 2025. Spill The Beans: Exploiting CPU Cache Side-Channels to Leak Tokens from Large Language Models. *arXiv preprint arXiv:2505.00817* (2025).
- [2] Atilla Akkus, Masoud Poorghaffar Aghdam, Mingjie Li, Junjie Chu, Michael Backes, Yuyang Zhang, and Sinem Sav. 2025. Generated data with fake privacy: Hidden dangers of fine-tuning large language models on generated data. In *34th USENIX Security Symposium (USENIX Security 25)*. 8075–8093.
- [3] Tyler Allen and Rong Ge. 2021. In-depth analyses of unified virtual memory system for GPU accelerated computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.
- [4] Bochuan Cao, Changjiang Li, Yuanpu Cao, Yameng Ge, Ting Wang, and Jinghui Chen. 2025. You Can’t Steal Nothing: Mitigating Prompt Leakages in LLMs via System Vectors. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 4423–4437.
- [5] Han Cho, Dongjun Kim, Seung-Eon Hwang, and Jongsun Park. 2024. SpARC: Token Similarity-Aware Sparse Attention Transformer Accelerator via Row-wise Clustering. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [6] Kexin Chu, Zecheng Lin, Dawei Xiang, Zixu Shen, Jianchang Su, Cheng Chu, Yiwei Yang, Wenhui Zhang, Wenfei Wu, and Wei Zhang. 2025. Selective kv-cache sharing to mitigate timing side-channels in LLM inference. *arXiv preprint arXiv:2508.08438* (2025).
- [7] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
- [8] Badhan Chandra Das, M Hadi Amini, and Yanzhao Wu. 2025. Security and privacy challenges of large language models: A survey. *Comput. Surveys* 57, 6 (2025), 1–39.
- [9] Debeshee Das, Jie Zhang, and Florian Trantèr. 2025. Blind baselines beat membership inference attacks for foundation models. In *2025 IEEE Security and Privacy Workshops (SPW)*. IEEE, 118–125.
- [10] Aditya Desai, Shuo Yang, Alejandro Cuadron, Matei Zaharia, Joseph E Gonzalez, and Ion Stoica. 2025. HashAttention: Semantic Sparsity for Faster Inference. In *Forty-second International Conference on Machine Learning*.
- [11] Hugging Face. 2024. Legal Q&A Dataset v1. <https://huggingface.co/datasets/dzunggg/legal-qa-v1>.
- [12] Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Peiyuan Zhou, Jiaxing Qi, Junjie Lai, Hayden Kwok-Hay So, Ting Cao, Fan Yang, et al. 2024. Seerattention: Learning intrinsic sparse attention in your LLMs. *arXiv preprint arXiv:2410.13276* (2024).
- [13] Zibo Gao, Junjie Hu, Feng Guo, Yixin Zhang, Yinglong Han, Siyuan Liu, Haiyang Li, and Zhiqiang Lv. 2025. I Know What You Said: Unveiling Hardware Cache Side-Channels in Local Large Language Model Inference. *34rd USENIX Security Symposium (USENIX Security 25)* (2025).
- [14] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [15] Yu He, Boheng Li, Liu Liu, Zhongjie Ba, Wei Dong, Yiming Li, Zhan Qin, Kui Ren, and Chun Chen. 2025. Towards label-only membership inference attack against pre-trained large language models. In *USENIX Security*.
- [16] Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. 2024. Pleak: Prompt leaking attacks against large language model applications. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3600–3614.
- [17] Siwon Kim, Sangdoo Yun, Hwaran Lee, Martin Gubri, Sungroh Yoon, and Seong Joon Oh. 2023. Propile: Probing privacy leakage in large language models. *Advances in Neural Information Processing Systems* 36 (2023), 20750–20762.
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [19] Lukas Lamster, Fabian Rauscher, Martin Unterguggenberger, and Stefan Mangard. 2025. WaitWatcher and WaitGuard: Detecting Flush-Based Cache Side-Channels Through Spurious Wakeups. In *European Symposium on Research in Computer Security*. Springer, 303–322.
- [20] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 155–172.

- [21] Dacheng Li, Rulin Shao, Anze Xie, Ying Sheng, Lianmin Zheng, Joseph Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. 2023. How long can context length of open-source llms truly promise?. In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*.
- [22] Ruixuan Liu, Tianhao Wang, Yang Cao, and Li Xiong. 2024. Precarious: How innocent pre-trained language models turn into privacy traps. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3511–3524.
- [23] Yupei Liu, Yuqi Jia, Jinyuan Jia, and Neil Zhenqiang Gong. 2025. Evaluating LLM-based Personal Information Extraction and Countermeasures. In *34th USENIX Security Symposium (USENIX Security 25)*. 1669–1688.
- [24] Nils Lukas, Ahmed Salem, Robert Sim, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2023. Analyzing leakage of personally identifiable information in language models. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 346–363.
- [25] Xinjian Luo, Ting Yu, and Xiaokui Xiao. 2025. Prompt inference attack on distributed large language model inference frameworks. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 1739–1753.
- [26] Zhifan Luo, Shuo Shao, Su Zhang, Lijing Zhou, Yuke Hu, Chenxu Zhao, Zhihao Liu, and Zhan Qin. 2025. Shadow in the cache: Unveiling and mitigating privacy risks of kv-cache in llm inference. *arXiv preprint arXiv:2508.09442* (2025).
- [27] Lingjuan Lyu, Xuanli He, and Yitong Li. 2020. Differentially private representation for NLP: Formal guarantee and an empirical study on privacy and fairness. *arXiv preprint arXiv:2010.01285* (2020).
- [28] Geoff McDonald and Jonathan Bar Or. 2025. Whisper Leak: a side-channel attack on Large Language Models. *arXiv preprint arXiv:2511.03675* (2025).
- [29] Matthieu Meeus, Shubham Jain, Marek Rei, and Yves-Alexandre de Montjoye. 2024. Did the neurons read your book? document-level membership inference for large language models. In *33rd USENIX Security Symposium (USENIX Security 24)*. 2369–2385.
- [30] Aastha Mehta, Mohamed Alzayat, Roberta De Viti, Björn B Brandenburg, Peter Druschel, and Deepak Garg. 2022. Pacer: Comprehensive Network {Side-Channel} Mitigation in the Cloud. In *31st USENIX Security Symposium (USENIX Security 22)*. 2819–2838.
- [31] Nurlan Nazaraliyev, Elaheh Sadredini, and Nael Abu-Ghazaleh. 2024. GPUVM: GPU-driven unified virtual memory. *arXiv preprint arXiv:2411.05309* (2024).
- [32] Nvidia. 2025. Nvidia Nsight. <https://developer.nvidia.com/nsight-systems>.
- [33] Xudong Pan, Mi Zhang, Shouling Ji, and Min Yang. 2020. Privacy risks of general-purpose language models. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1314–1331.
- [34] Pratyush Patel, Esha Chouksey, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [35] Prasad. 2022. Healthcare Dataset. <https://www.kaggle.com/datasets/prasad22/healthcare-dataset>.
- [36] Yousef Saeedian. 2023. Financial Q&A 10K. <https://www.kaggle.com/datasets/yousefsaeedian/financial-q-and-a-10k>.
- [37] Linke Song, Zixuan Pang, Wenhao Wang, Zihao Wang, XiaoFeng Wang, Hongbo Chen, Wei Song, Yier Jin, Dan Meng, and Rui Hou. 2025. The early bird catches the leak: Unveiling timing side channels in llm serving systems. *IEEE Transactions on Information Forensics and Security* (2025).
- [38] Yutao Sun, Zhenyu Li, Yike Zhang, Tengyu Pan, Bowen Dong, Yuyi Guo, and Jianyong Wang. 2025. Efficient attention mechanisms for large language models: A survey. *arXiv preprint arXiv:2507.19595* (2025).
- [39] Yicong Tan, Xinyue Shen, Yun Shen, Michael Backes, and Yang Zhang. 2025. On the Effectiveness of Prompt Stealing Attacks on In-The-Wild Prompts. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 392–410.
- [40] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. QUEST: query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning*. 47901–47911.
- [41] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295* (2024).
- [42] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [43] Marcos Treviso, António Góis, Patrick Fernandes, Erick Fonseca, and André FT Martins. 2022. Predicting attention sparsity in transformers. In *Proceedings of the Sixth Workshop on Structured Prediction for NLP*. 67–81.
- [44] Muhammad Umar, Akhilesh Parag Marathe, Monami Dutta Gupta, Shubham Jogprakash Ghosh, G Edward Suh, and Wenjie Xiong. 2025. Efficient Memory Side-Channel Protection for Embedding Generation in Machine Learning. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 423–441.
- [45] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [46] Fei Wang and Baochun Li. 2025. Leaner Training, Lower Leakage: Revisiting Memorization in LLM Fine-Tuning with LoRA. *arXiv preprint arXiv:2506.20856* (2025).
- [47] Kai Wang, Fengkai Yuan, Rui Hou, Jingqiang Lin, Zhenzhou Ji, and Dan Meng. 2019. CacheGuard: a security-enhanced directory architecture against continuous attacks. In *Proceedings of the 16th ACM International Conference on Computing Frontiers*. 32–41.
- [48] Roy Weiss, Daniel Ayzenshteyn, and Yisroel Mirsky. 2024. What was your prompt? a remote keylogging attack on AI assistants. In *33rd USENIX Security Symposium (USENIX Security 24)*. 3367–3384.
- [49] Rui Wen, Zheng Li, Michael Backes, and Yang Zhang. 2024. Membership inference attacks against in-context learning. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3481–3495.
- [50] Guanlong Wu, Zheng Zhang, Yao Zhang, Weili Wang, Jianyu Niu, Ye Wu, and Yinqian Zhang. 2025. I know what you asked: Prompt leakage via kv-cache sharing in multi-tenant llm serving. In *Proceedings of the 2025 Network and Distributed System Security (NDSS) Symposium*. San Diego, CA, USA.
- [51] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Shang Yang, Haotian Tang, Yao Fu, Song Han, et al. 2025. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. In *The Thirteenth International Conference on Learning Representations*.
- [52] Jiaming Xu, Jiayi Pan, Hanzhen Wang, Yongkang Zhou, Jiancai Ye, Yu Wang, and Guohao Dai. 2025. SpeContext: Enabling Efficient Long-context Reasoning with Speculative Context Sparsity in LLMs. *arXiv preprint arXiv:2512.00722* (2025).
- [53] Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. 2025. Xattention: Block sparse attention with antidiagonal scoring. *arXiv preprint arXiv:2503.16428* (2025).
- [54] Biwei Yan, Kun Li, Minghui Xu, Yueyan Dong, Yue Zhang, Zhaochun Ren, and Xiuzhen Cheng. 2024. On protecting the data privacy of large language models (llms): A survey. *arXiv preprint arXiv:2403.05156*

- (2024).
- [55] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
 - [56] Yong Yang, Changjiang Li, Qingming Li, Oubo Ma, Haoyu Wang, Zonghui Wang, Yandong Gao, Wenzhi Chen, and Shouling Ji. 2025. PRSA: Prompt Stealing Attacks against Real-World Prompt Services. In *34th USENIX security symposium (USENIX Security 25)*. 2283–2302.
 - [57] Tianchen Zhang, Gururaj Saileshwar, and David Lie. 2024. Time will tell: Timing side channels via output token count in large language models. *arXiv preprint arXiv:2412.15431* (2024).
 - [58] Zhenkai Zhang, Tyler Allen, Fan Yao, Xing Gao, and Rong Ge. 2023. TunnelS for Bootlegging: Fully Reverse-Engineering GPU TLBs for Challenging Isolation Guarantees of NVIDIA MIG. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 960–974.
 - [59] Zhenkai Zhang, Kunbei Cai, Yanan Guo, Fan Yao, and Xing Gao. 2024. Invalidate+Compare: A Timer-Free GPU Cache Attack Primitive. In *33rd USENIX Security Symposium (USENIX Security 24)*. 2101–2118.
 - [60] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems* 36 (2023), 34661–34710.
 - [61] Zheng Zhang, Guanlong Wu, Sen Deng, Shuai Wang, and Yinqian Zhang. 2025. NetEcho: From Real-World Streaming Side-Channels to Full LLM Conversation Recovery. *arXiv preprint arXiv:2510.25472* (2025).
 - [62] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.
 - [63] Xinyao Zheng, Husheng Han, Shangyi Shi, Qiyang Fang, Zidong Du, Xing Hu, and Qi Guo. 2024. Inputsntch: Stealing input in llm services via timing side-channel attacks. *arXiv preprint arXiv:2411.18191* (2024).
 - [64] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
 - [65] Qianchao Zhu, Jiangfei Duan, Chang Chen, Siran Liu, Xiuhong Li, Guanyu Feng, Xin Lv, Xiao Chuanfu, Dahua Lin, and Chao Yang. 2025. Sampleattention: Near-lossless acceleration of long context llm inference with adaptive structured sparse attention. *Proceedings of Machine Learning and Systems* 7 (2025).